Christopher Raúl Gómez Álvarez

7183403

Gestión de Proyectos de Inteligencia Artificial

1-11-2026

Fase II

1. Introducción y relación con Fase I

En la Fase I se planteó la implementación de un asistente de inteligencia artificial que ayudará a responder preguntas administrativas frecuentes y a canalizar solicitudes de cita en Clínica Nakashima. El alcance dejó fuera las recomendaciones de tratamientos y los diagnósticos, ya que estas actividades requieren intervención profesional. Para esta segunda fase desarrollé un prototipo local orientado a demostrar cómo integrar un modelo entrenado en una aplicación web y cómo llevarlo a un contenedor reproducible.

Para mantener el ejercicio dentro de un alcance manejable, se utilizó un clasificador de intenciones en lugar de un modelo generativo. Este clasificador identifica si un mensaje trata sobre horarios, ubicaciones, servicios, precios, citas, saludos o consultas clínicas. Posteriormente, el sistema recupera una respuesta administrativa previamente definida o solicita intervención humana.

2. Arquitectura y herramientas

El prototipo cuenta con una interfaz web creada con HTML, CSS y JavaScript. El frontend envía la consulta mediante una solicitud POST a la API REST implementada en

FastAPI. El backend valida el mensaje e invoca un modelo de clasificación entrenado con scikit-learn, utilizando TF-IDF de caracteres y regresión logística. Para las preguntas clínicas se incorporó una regla de seguridad que prioriza la derivación humana. El modelo y sus dependencias se integraron en una imagen definida mediante Dockerfile, con Python 3.11 y ejecución a través de Uvicorn.



Figura 1. Arquitectura general del prototipo Nakashima IA.

El conjunto de entrenamiento contiene 154 preguntas sintéticas distribuidas en siete categorías. Se utilizó una partición estratificada de 75 % para entrenamiento y 25 % para prueba de evaluación, con semilla 42. Después de obtener las métricas, se ajustó un artefacto final con todos los ejemplos para incorporarlo a la API. Los datos se generaron únicamente para la actividad, sin utilizar mensajes reales ni expedientes clínicos.

3. Pruebas y validación

En la evaluación retenida, el clasificador obtuvo una exactitud de 84.62 % y un F1 macro de 84.29 % sobre 39 ejemplos sintéticos. Como referencia, un clasificador que siempre predice la clase más frecuente obtuvo 12.82 % de exactitud. La comparación muestra que el modelo extrae patrones del vocabulario incluido en los ejemplos, pero estas cifras no representan desempeño con pacientes reales ni uso clínico. La matriz de confusión permite identificar errores: de seis

consultas sintéticas de prueba sobre costos, cuatro se clasificaron correctamente y dos se confundieron con servicios; de seis preguntas de horarios, cuatro fueron correctas, una se confundió con citas y otra con consulta clínica. Estos errores sugieren que hacen falta más ejemplos de preguntas ambiguas y que una respuesta automática debe tener mecanismos de derivación.

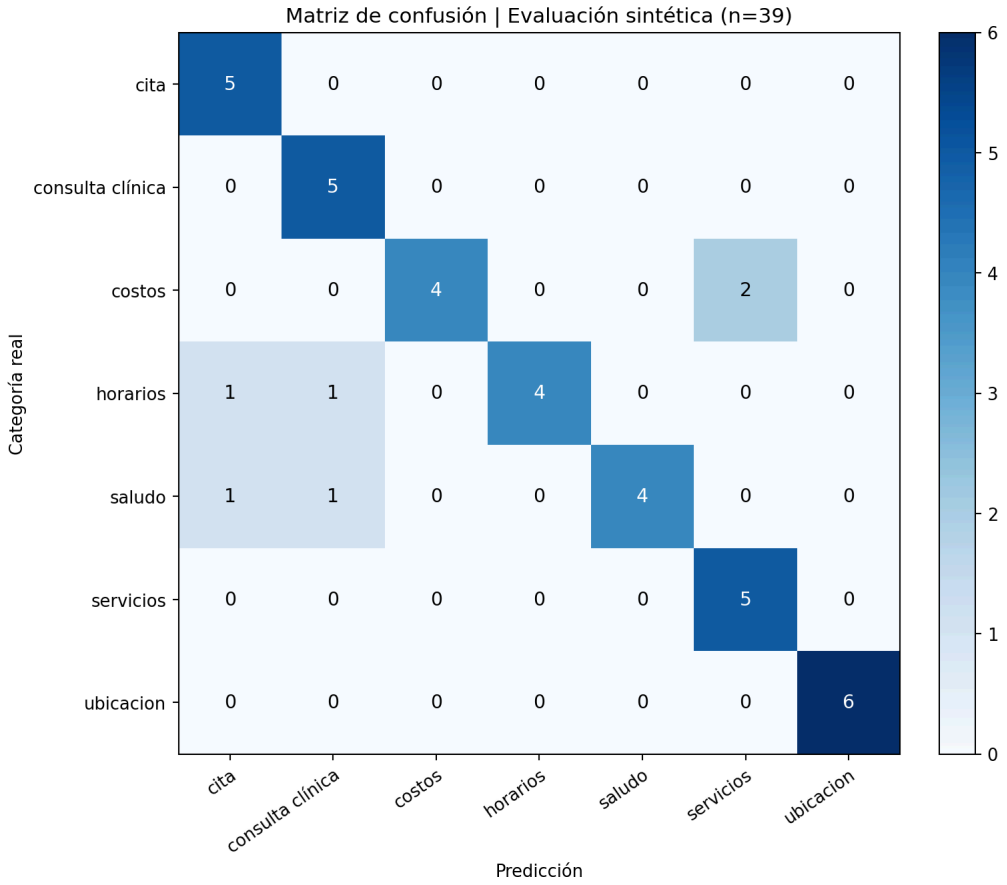


Figura 2. Matriz de confusion

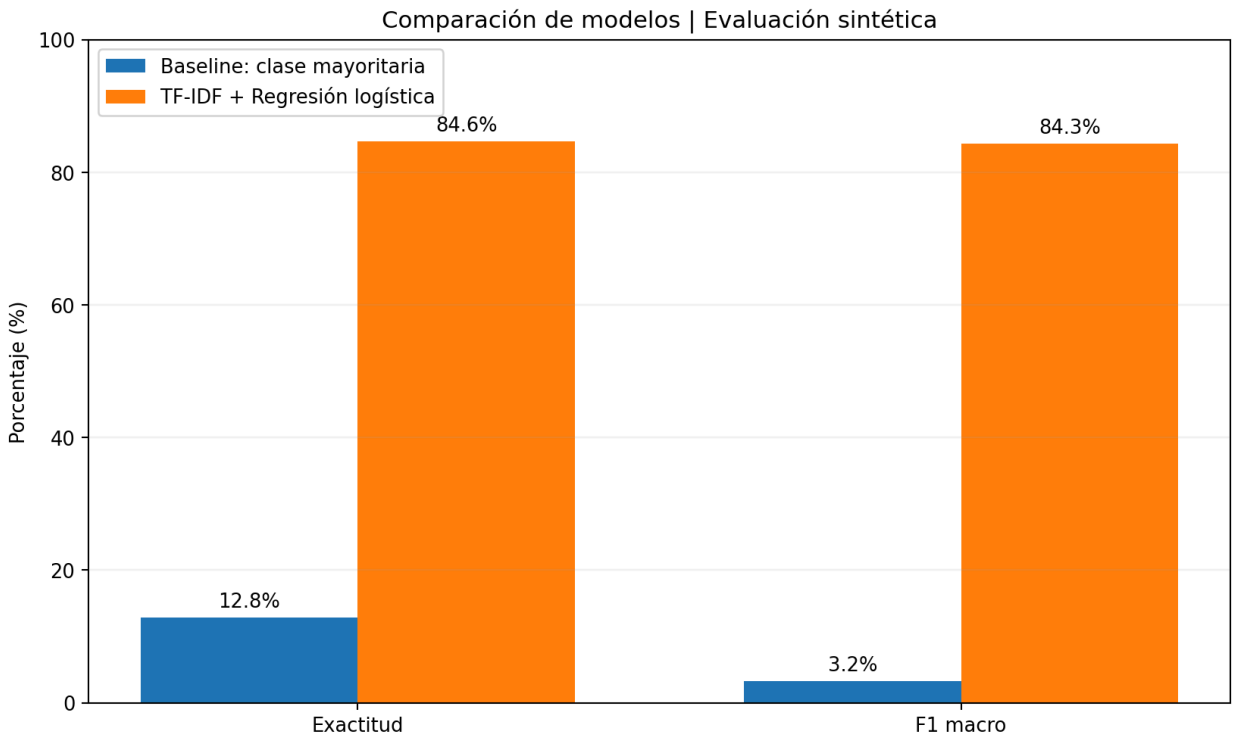


Figura 3: Comparación de Modelos

Las pruebas automatizadas ejecutadas localmente con Python 3.13 finalizaron con 20 casos aprobados. Posteriormente, se validó el funcionamiento de la aplicación dentro del contenedor Docker con Python 3.11, mediante pruebas manuales de sus endpoints y de la interfaz web. La ejecución de la suite completa de pytest dentro del contenedor queda como una validación adicional.

Caso de prueba	Código HTTP	Resultado
Solicitud de cita	200	Clasificación cita, confianza 78.39%
Consulta por dolor dental	200	Derivación clínica mediante regla de seguridad
Pregunta fuera de dominio	200	Categoría desconocida, confianza 33.13%
Mensaje vacío	422	Rechazado por longitud insuficiente
Más de 500 caracteres	422	Rechazado por exceder la longitud máxima

Las pruebas ejecutadas desde Swagger UI permitieron evaluar no solo la clasificación de mensajes, sino también la validación de datos y la respuesta del sistema ante situaciones fuera de su alcance. Los cinco escenarios produjeron los resultados esperados.

Destaca la prueba de seguridad clínica, donde el sistema utilizó una regla específica para derivar al usuario hacia atención profesional sin emitir un diagnóstico. Por otro lado, las respuestas HTTP 422 confirmaron que la API rechaza mensajes vacíos o superiores a 500 caracteres.

Aunque los resultados fueron satisfactorios para los casos seleccionados, estos no garantizan un funcionamiento correcto ante todas las consultas posibles. Se requiere ampliar las pruebas con mensajes de distintas estructuras y expresiones antes de considerar su uso operativo.

4. Contenerización y despliegue local

Se preparó un Dockerfile a partir de Python 3.11 slim, que instala dependencias, copia el frontend, la API y el artefacto entrenado, ejecuta el servicio con un usuario no privilegiado, expone el puerto 8000 y habilita una comprobación de salud. Con ello, la aplicación cuenta con una configuración replicable. El manual README documenta los comandos de construcción, ejecución y validación del contenedor, así como las rutas /, /health, /docs y /api/consultar

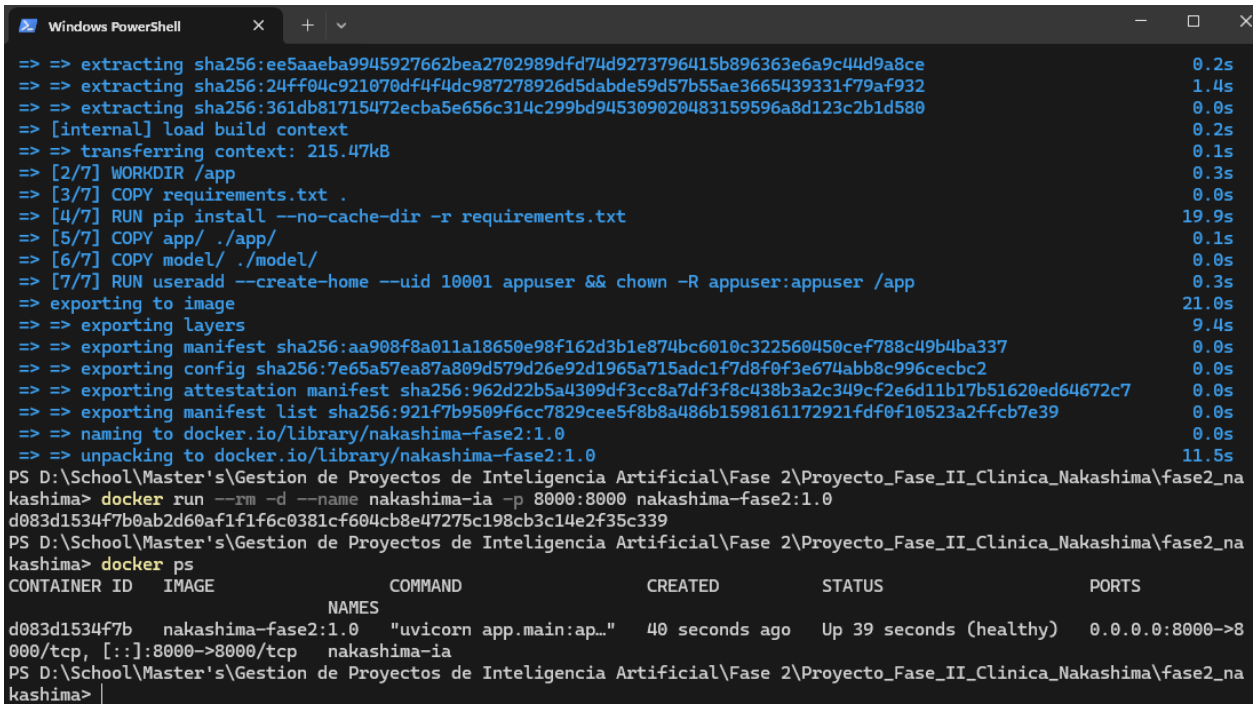


Figura 4: Ejecución del contenedor y verificación de su estado en Docker

Para verificar el despliegue local, construí la imagen Docker mediante el comando docker build -t nakashima-fase2:1.0 . y ejecuté el contenedor con docker run --rm -d --name nakashima-ia -p 8000:8000 nakashima-fase2:1.0.

La ejecución se completó correctamente y el comando docker ps confirmó que el contenedor permanecía activo, con estado saludable y el puerto 8000 disponible. Posteriormente, ingresé desde el navegador a las rutas /, /health y /docs.

La consulta al endpoint /health devolvió el estado ok y confirmó que el modelo estaba cargado. También probé el asistente desde su interfaz web y mediante Swagger UI, comprobando la comunicación entre el frontend, el backend y el clasificador. Estas pruebas permitieron verificar el funcionamiento de la aplicación dentro del contenedor, utilizando Python 3.11.

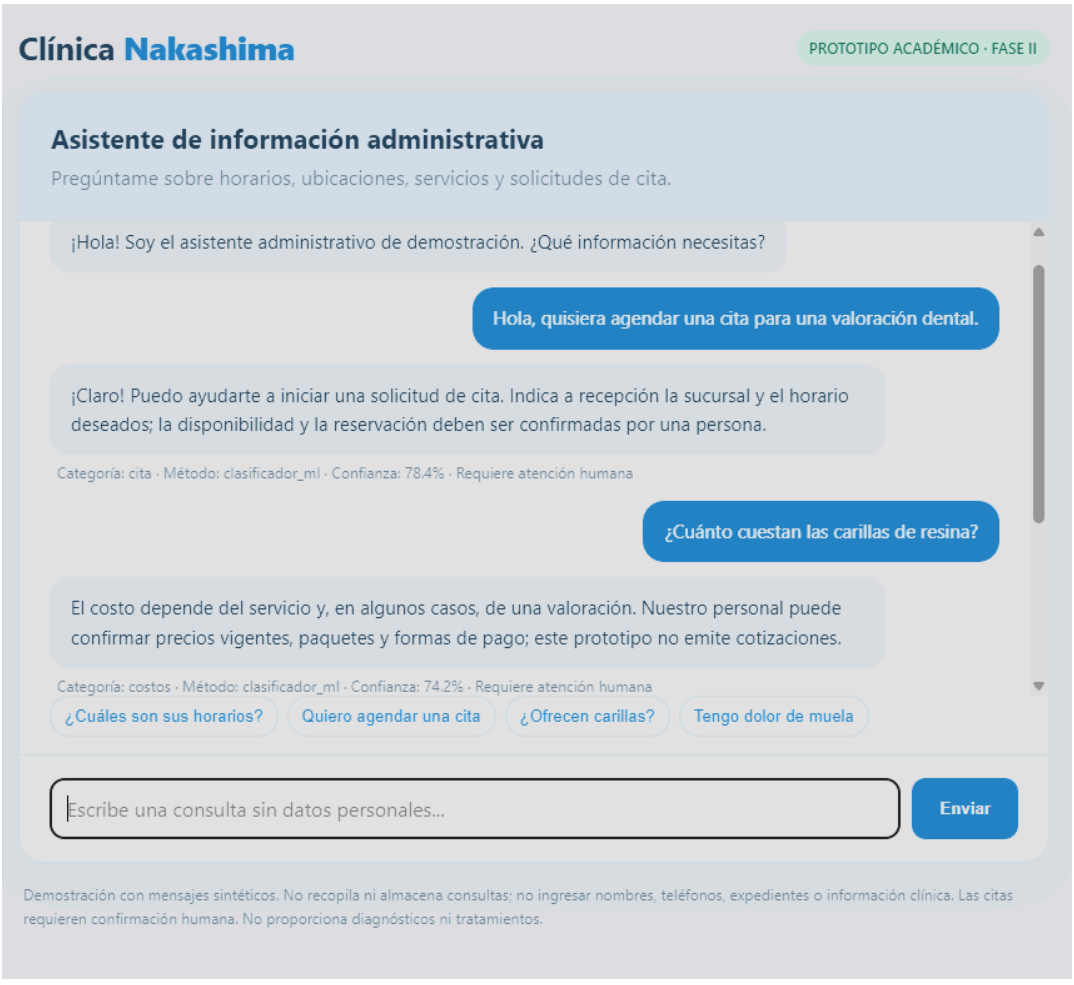


Figura 5: Interacción entre el usuario y el asistente mediante el frontend.

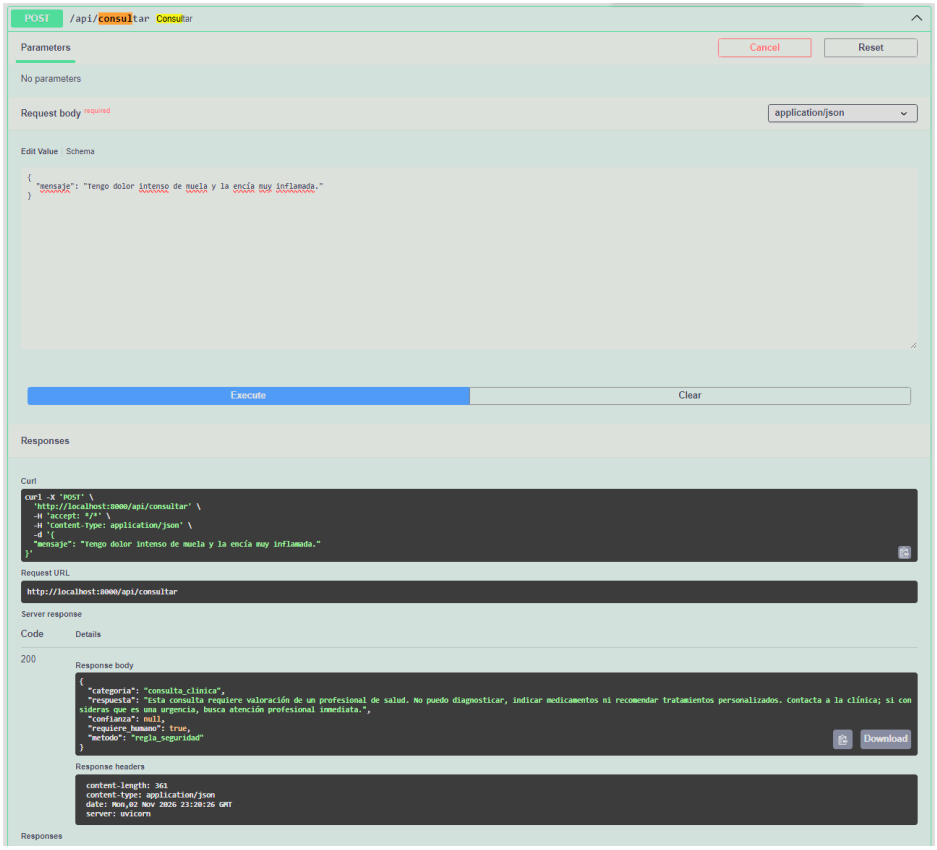


Figura 6. Respuesta de la API ante una consulta clínica que requiere derivación humana.

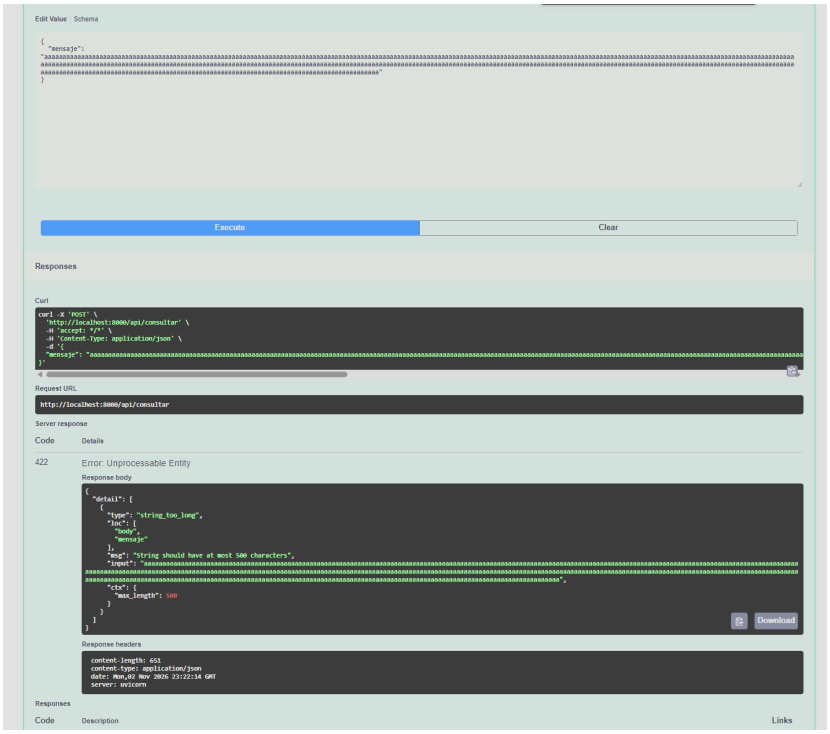


Figura 7. Validación de entradas inválidas mediante respuestas HTTP 422.

5. Limitaciones, riesgos y conclusiones

El modelo se entrenó con un conjunto pequeño y artificial, por lo que todavía puede clasificar incorrectamente preguntas ambiguas, vocabulario no visto y consultas que combinan varias intenciones. El umbral de confianza es exploratorio y no ha sido calibrado con información real. Las reglas de seguridad tampoco cubren todas las formas posibles de describir un problema dental. Por esas razones, el prototipo no debe usarse para dar consejos clínicos, administrar datos sensibles o confirmar citas automáticamente.

Este ejercicio permitió integrar los componentes de una aplicación de IA de principio a fin: entrenamiento, archivo de modelo, API, frontend, validaciones automatizadas y preparación del contenedor. Para una versión operativa se necesitarían pruebas con datos autorizados y

desidentificados, mejores mecanismos de privacidad, controles de acceso, monitoreo, actualización de contenidos y revisión profesional de las respuestas. En términos de gestión del proyecto, esta segunda fase ayuda a identificar qué elementos técnicos funcionan y qué aspectos aún requieren validación antes de pasar a producción.

6. Repositorio y reproducibilidad

El proyecto se organizó en un repositorio que incluye el código fuente del frontend y backend, el modelo entrenado, los archivos de entrenamiento, las pruebas automatizadas y las instrucciones para ejecutar la aplicación.

El archivo Dockerfile permite construir la imagen del sistema, mientras que requirements.txt especifica las dependencias necesarias. Por su parte, el archivo README.md contiene las instrucciones de instalación, ejecución y verificación mediante los endpoints /, /health, /docs y /api/consultar.

Se agregó también una carpeta de evidencias con capturas de la ejecución local, las respuestas de la API y las validaciones realizadas. Esta organización facilita la revisión del proyecto y permite reproducir las pruebas en otro equipo que tenga Docker correctamente configurado.

Repositorio de GitHub: https://github.com/GomezAlvarezCR/Fase-II---Christopher-Gomez.

Referencias

Universidad Tecmilenio. (s. f.). Tema 15: Despliegue local [Material de clase].

Universidad Tecmilenio. (s. f.). Tema 16: Despliegue en la nube [Material de clase].

Universidad Tecmilenio. (s. f.). Rúbrica de evaluación: Proyecto fase 2 [Material de clase].

ISO/IEC. (2022). ISO/IEC 23053:2022 — Framework for Artificial Intelligence (AI) Systems Using Machine Learning (ML).